

Highlights

The Beggining of Control Revolution: Offset-free Koopman MPC

Patrik Valábek

- Research highlight 1
- Research highlight 2

The Beggining of Control Revolution: Offset-free Koopman MPC

Patrik Valábek

^a*STU, Redlinskeho 9, Bratislava, Slovakia*

Abstract

Keywords:

1. Introduction

2. Preliminaries and Notation

2.1. Offset-Free MPC

In this subsection, we will present a standard offset free model predictive control framework, introduced by Pannocchia and Rawlings (2003). This approach consist of state estimation - observer, target optimization and dynamic optimization. The whole approach is used to introduce integral action into model predictive control to help mitigate plant-model mismatch and ensure offset free control. If this compensation would not be used, the plant with this controller would not reach a desired reference (other than at origin), due to predictions using different model than actual plant.

The offset-free augmented system can be written in the following state-space form, including the disturbance input matrix B_d and disturbance output matrix C_d :

$$\begin{bmatrix} x_{k+1} \\ d_{k+1} \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} \begin{bmatrix} x_k \\ d_k \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_k, \quad y_k = \begin{bmatrix} C & C_d \end{bmatrix} \begin{bmatrix} x_k \\ d_k \end{bmatrix} \quad (1)$$

where x_k is the original system state, d_k is the estimated constant disturbance, u_k is the control input, y_k is the output, A , B , and C are the original state-space matrices, B_d and C_d encode how the disturbance enters the state and output equations, and I is the identity matrix. Here, n_x , n_u , n_y , and n_d denote the state, input, output, and disturbance dimensions, respectively, so

that $x_k \in \mathbb{R}^{n_x}$, $u_k \in \mathbb{R}^{n_u}$, $y_k \in \mathbb{R}^{n_y}$, $d_k \in \mathbb{R}^{n_d}$, $A \in \mathbb{R}^{n_x \times n_x}$, $B \in \mathbb{R}^{n_x \times n_u}$, $C \in \mathbb{R}^{n_y \times n_x}$, $B_d \in \mathbb{R}^{n_x \times n_d}$, $C_d \in \mathbb{R}^{n_y \times n_d}$, and $I \in \mathbb{R}^{n_d \times n_d}$. For the offset free behavior, the number of the disturbances n_d must be lower or equal to the number of the measurements n_y . Also it is necessary to note that the number of offset free tracking is achievable only in the number of disturbances that are estimated. For the system where $n_d = n_y$ system and choice of the matrices B_d and C_d there is a necessary condition, that:

$$\text{rank} \begin{bmatrix} I - A & -B_d \\ C & C_d \end{bmatrix} = n_x + n_y. \quad (2)$$

When solving the MPC, both states and disturbance are approximated by observer.

2.2. Koopman Operator

The koopman operator theory was introduced by Koopman (1931) and further popularize by Mezić (2005). The theory enables to exactly model nonlinear systems linearly, by lifting its current states/measurements into infinite dimensional variables, where the governing operator for the evolving system is linear.

TODO: fancy smenši equation

Over the years many approximations of the koopman operator from the data were proposed, starting with connection of DMD to Koopman operator theory by Rowley et al. (2009), extended to the EDMD by Williams et al. (2015) and recently the neural networks were used to best approximate the lifting functions so the system behaves linearly. The algorithms were as well developed not only for autonomous systems but for the systems with control as well Kaiser et al. (2021) **TODO: elaborate half a sentence** . Koopman operator met the model predictive control in work of Korda and Mezić (2018), and in recent years almost every kind of MPC was transformed to accept some kind of Koopman Operator, mostly represented as an LTI system in the state space form. The offset free version is no exception, in the work of **TODO: Zotero, kukni papere, prosím ešte aj naskenuj ďalšie** .

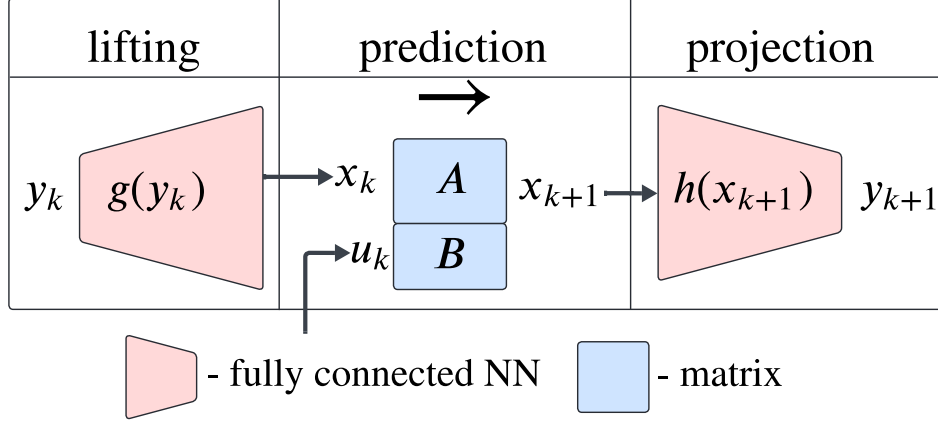


Figure 1: TODO:

For the purpose of this work we are using the Deep Koopman Approximation introduced by Lusch et al. (2018), transformed for the nonautonomous systems, as in Fig. 1. The control inputs u are used in the scheme directly, without any transformation to preserve the structure necessary for the use in MPC. We use two kinds of approximations of Koopman operator. Firstly the LTI approximation, where the projection function is linear (matrix C), as in Fig. 2 and second the approximation, where only the lifted system evolves linearly, as already shown in Fig. 1, but its projection is nonlinear (as it should be from the definition of Koopman operator).

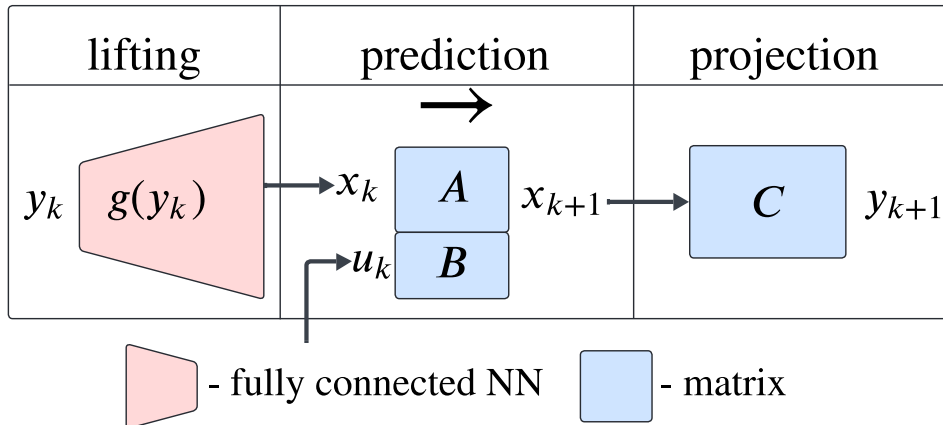


Figure 2: TODO:

3. Koopman MPC with Offset-Free

TODO: This heading is incomplete and is in title case; headings must be sentence case. Confirm the intended title, for example "Offset-free Koopman MPC", and it will be applied.

In this subsection, we will present a standard offset free model predictive control framework using approximation of Koopman operator as a linear time invariant (LTI) model. As observer is standard Kalman filter, we will discuss mainly remaining two optimization problems. As we use the LTI approximation of Koopman operator, we can use the standard MPC formulation with linear constraints and objective function. The control structure is shown in Fig. 3.

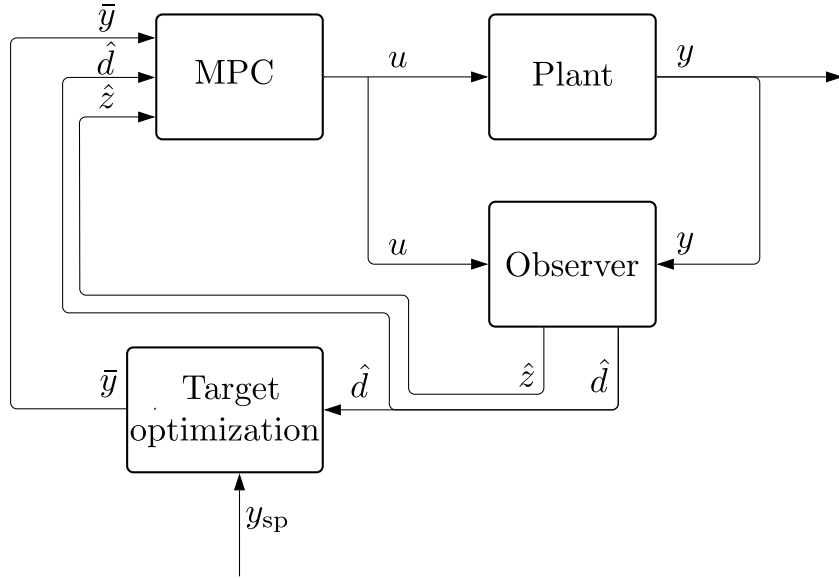


Figure 3: Basic closed-loop control structure.

The target optimization problem computes the steady-state targets $\bar{u}$, $\bar{z}$, $\bar{y}$ that minimizes the difference between reference y_{ref} and feasible output $\bar{y}$ in steady state. The problem is formulated as:

$$\min_{\bar{u}, \bar{z}, \bar{y}} (\bar{y} - y_{\text{ref}})^\top Q_y^t (\bar{y} - y_{\text{ref}}), \quad (3a)$$

$$\text{s.t.} \quad \bar{z} = A\bar{z} + B\bar{u}, \quad (3b)$$

$$\bar{y} = C\bar{z} + d, \quad (3c)$$

$$u_{\min} \leq \bar{u} \leq u_{\max}, \quad (3d)$$

$$y_{\min} \leq \bar{y} \leq y_{\max}, \quad (3e)$$

$$d = \hat{d}(t), \quad y_{\text{ref}} = y_{\text{sp}}(t), \quad (3f)$$

where $\bar{u} \in \mathbb{R}^{n_u}$, $\bar{z} \in \mathbb{R}^{n_z}$, and $\bar{y} \in \mathbb{R}^{n_y}$ are the steady-state input, lifted state, and output, respectively; $Q_y^t \succ 0$ is the output weight of the target optimization, which is tuned separately from the weights of the MPC problem; $A \in \mathbb{R}^{n_z \times n_z}$, $B \in \mathbb{R}^{n_z \times n_u}$, $C \in \mathbb{R}^{n_y \times n_z}$ are the state-space matrices of the LTI Koopman model; $d = \hat{d}(t) \in \mathbb{R}^{n_y}$ is the additive output disturbance estimate provided by the Kalman filter; $y_{\text{ref}} = y_{\text{sp}}(t)$ is the output setpoint; and $u_{\min}, u_{\max}, y_{\min}, y_{\max}$ are the input and output constraint bounds.

Remark 1. For this **TODO: This remark is empty. What should it state?**
A likely candidate is the condition under which the target problem stays feasible when $n_d = n_y$, which would follow the rank condition (2).

Given the steady-state target $(\bar{u}, \bar{z}, \bar{y})$, the MPC problem is solved at each sampling instant to compute the optimal input sequence over the horizon N :

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} [(y_k - \bar{y})^\top Q_y (y_k - \bar{y}) + \Delta u_k^\top Q_{\Delta u} \Delta u_k + (u_k - \bar{u})^\top Q_u (u_k - \bar{u})], \quad (4a)$$

$$\text{s.t.} \quad z_{k+1} = Az_k + Bu_k, \quad k \in \mathbb{N}_0^{N-1}, \quad (4b)$$

$$y_k = Cz_k + d, \quad k \in \mathbb{N}_0^{N-1}, \quad (4c)$$

$$\Delta u_k = u_k - u_{k-1}, \quad k \in \mathbb{N}_0^{N-1}, \quad (4d)$$

$$u_{\min} \leq u_k \leq u_{\max}, \quad k \in \mathbb{N}_0^{N-1}, \quad (4e)$$

$$y_{\min} \leq y_k \leq y_{\max}, \quad k \in \mathbb{N}_0^{N-1}, \quad (4f)$$

$$z_0 = \hat{z}(t), \quad d = \hat{d}(t), \quad u_{-1} = u(t - T_s), \quad (4g)$$

where $Q_y \succ 0$ is the output tracking weight matrix, $Q_{\Delta u} \succ 0$ the input increment weight matrix, and $Q_u \succeq 0$ the weight on the deviation of the

input from its steady-state target; $\hat{z}(t) \in \mathbb{R}^{n_z}$ is the lifted state estimate provided by the Kalman filter; and $u_{-1} = u(t - T_s)$ is the input applied at the previous sampling instant, used to compute the first increment Δu_0 . The first term of the objective function (4a) penalizes the predicted tracking error with respect to the output target, the second term penalizes the movement of the manipulated variables, and the third term keeps the inputs at the steady-state target computed by (3), which resolves the remaining degrees of freedom when the plant has more inputs than outputs. Following the receding-horizon principle, only the first element u_0^* of the optimal sequence is applied to the plant.

4. Methodology

In this section, we present the proposed offset-free Koopman MPC framework. The main drawback of the current Koopman MPC formulations, whether offset-free, robust, or of any other kind, is that the optimization is carried out without fully exploiting the knowledge of the lifted space. The prediction is made in the lifted space, but the objective function is expressed in the original space, which is recovered from the lifted state by the linear transformation $y_k = Cz_k$. This transformation is nonlinear by nature, $y_k = h(z_k)$, and it is restricted to a linear one only so that a linear MPC problem is obtained. Formulations that do not use this transformation directly usually rely on an equivalent one, for example by augmenting the lifted state with the measurements, in which case the recovery of the measurements from the lifted state is again linear. **TODO: This sentence characterises other authors' formulations but carries no citation, and the style skill forbids attributing a claim to an uncited source. Which papers augment the lifted state with the measurements? Supply the keys and they will be cited here, one reference per claim.** We show that the lifted space can be used directly in the objective function, and we derive the corresponding transformation of the weighting matrix. The back transformation is thereby eliminated from the objective function of the MPC problem, while it is still required to compute the steady-state target and to impose the output constraints. In both of these, we approximate it by a first-order Taylor expansion.

4.1. Deep Koopman identification

TODO: Register clash inside one section: this subsection is impersonal ("the model is unrolled", "each candidate configuration is

trained”) while Sec. 4.2–4.7 are first-person plural (“we propose”, “we encountered”). The style skill allows either register but forbids mixing them within a section. Decide which one holds for the whole Methodology, and the other block will be converted.

Both prediction models used in this work are obtained with the deep Koopman architecture of Fig. 1, in which the encoder lifts the measured outputs, the lifted state evolves linearly, and the decoder projects the lifted state back to the outputs. Given an input–output trajectory of the plant, the model is unrolled over a window of N_r samples as

$$z_0 = g(y_0), \quad (5a)$$

$$z_{i+1} = Az_i + Bu_i, \quad i \in \mathbb{N}_0^{N_r-1}, \quad (5b)$$

$$\hat{y}_i = h(z_i), \quad i \in \mathbb{N}_0^{N_r}, \quad (5c)$$

where N_r is the length of the training window, i is the sample index within this window, y_i and u_i are the measured output and the applied input taken from the data, $z_i \in \mathbb{R}^{n_z}$ is the lifted state, and $\hat{y}_i \in \mathbb{R}^{n_y}$ is the predicted output. The encoder g and the decoder h are multilayer perceptrons, while the input map B is kept linear, so that the identified model is an LTI system in the lifted coordinates and can be used directly in the optimization problems (8) and (9).

The four terms of the training loss are the multi-step output prediction error over the whole window, the consistency between the unrolled lifted state and the lifted measurement, the one-step output prediction error, and the reconstruction error of the autoencoder, resulting in

$$\begin{aligned} \mathcal{L} = & \frac{w_y}{N_r - 1} \sum_{i=1}^{N_r-1} \|\hat{y}_i - y_i\|_2^2 + \frac{w_z}{N_r - 1} \sum_{i=1}^{N_r-1} \|z_i - g(y_i) - Bu_i\|_2^2 \\ & + w_1 \|\hat{y}_1 - y_1\|_2^2 + w_r \|\hat{y}_0 - y_0\|_2^2, \end{aligned} \quad (6)$$

where $\mathcal{L}$ is the loss minimized during training and w_y , w_z , w_1 , and w_r are the weights of the four terms, respectively. All terms are additionally averaged over the minibatch of training windows. The one-step term w_1 is fixed to one, so that only three weights have to be chosen, and the remaining weights are scaled relative to it.

When the decoder is restricted to a single matrix, $h(z) = Cz$, the identified model is the LTI model of the standard offset-free Koopman MPC of Sec. 3. When the decoder is a nonlinear map, the model is the one used by the proposed scheme, and a constant output matrix is obtained afterwards by linear least squares from the lifted trajectory of the training data. This matrix is not used in the objective function or in the constraints of (9); it only provides the initial output matrix of the observer of Sec. 4.6, which is replaced by the Jacobian $H(z_{\text{LP}})$ at every subsequent sampling instant.

The lifted dimension n_z , the depth and the width of the encoder and the decoder, the activation function, the weights in (6), the window length N_r , the minibatch size, and the learning rate are not tuned by hand. Each candidate configuration is trained, the resulting model is inserted into the closed loop, and the achieved value of the closed-loop objective function is used as the score of the configuration; the search itself is carried out with the tree-structured Parzen estimator of Optuna Akiba et al. (2019). Scoring a model by the closed-loop performance rather than by the open-loop prediction error is a deliberate choice, because the two are only loosely related, as the results of Sec. 5 and Sec. 7 show. The same criterion is used to select the order and the horizons of the subspace model that serves as the baseline, so that no model is favored by the way its parameters were chosen. Note that this procedure is not a contribution of this work and any comparable model selection scheme can replace it.

4.2. Linearization of the output mapping

For the control problem, the transformation from the lifted space back to the original space cannot be eliminated completely, because both the steady-state target and the output constraints are expressed in the original variables. We therefore approximate the output mapping by a first-order Taylor expansion, and we consider two linearization points, the target computed at the previous sampling instant and the current state estimate, which gives

$$y_k = h(z_k) \approx h(\bar{z}(t - T_s)) + H(\bar{z}(t - T_s))(z_k - \bar{z}(t - T_s)), \quad (7a)$$

$$y_k = h(z_k) \approx h(\hat{z}(t)) + H(\hat{z}(t))(z_k - \hat{z}(t)), \quad (7b)$$

where $h : \mathbb{R}^{n_z} \rightarrow \mathbb{R}^{n_y}$ is the decoder of the identified model, which acts as the nonlinear output mapping; $H(\cdot) \in \mathbb{R}^{n_y \times n_z}$ is the Jacobian of h evaluated at its argument; and $\bar{z}(t - T_s)$ is the steady-state lifted state computed by the target optimizer at the previous sampling instant.

4.3. Target optimization

The objective function of the target optimization is the same as in the standard offset-free MPC of Sec. 3. The difference is in the output equation, where the linear map of (3c) is replaced by the linearization (7a) of the nonlinear output mapping. The proposed target optimization problem is therefore formulated as:

$$\min_{\bar{u}, \bar{z}, \bar{y}} (\bar{y} - y_{\text{ref}})^\top Q_y^t (\bar{y} - y_{\text{ref}}), \quad (8a)$$

$$\text{s.t.} \quad \bar{z} = A\bar{z} + B\bar{u}, \quad (8b)$$

$$\bar{y} = H(z_{\text{LP}})\bar{z} + h(z_{\text{LP}}) - H(z_{\text{LP}})z_{\text{LP}} + d, \quad (8c)$$

$$u_{\min} \leq \bar{u} \leq u_{\max}, \quad (8d)$$

$$y_{\min} \leq \bar{y} \leq y_{\max}, \quad (8e)$$

$$d = \hat{d}(t), \quad y_{\text{ref}} = y_{\text{sp}}(t), \quad z_{\text{LP}} = \bar{z}(t - T_s), \quad (8f)$$

where z_{LP} is the linearization point at which the output mapping is approximated by its first-order Taylor expansion. We propose to choose this point as close as possible to $\bar{z}$, and we therefore use the target $\bar{z}(t - T_s)$ computed at the previous sampling instant, which corresponds to the linearization (7a). This choice improves the accuracy of the approximation of the transformation from the lifted space to the original space and, consequently, the accuracy of the computed target.

4.4. MPC

The proposed MPC problem is formulated as:

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} [(z_k - \bar{z})^\top Q_z (z_k - \bar{z}) + \Delta u_k^\top Q_{\Delta u} \Delta u_k + (u_k - \bar{u})^\top Q_u (u_k - \bar{u})], \quad (9a)$$

$$\text{s.t.} \quad z_{k+1} = Az_k + Bu_k, \quad k \in \mathbb{N}_0^{N-1}, \quad (9b)$$

$$y_k = H(z_{\text{LP}})z_k + h(z_{\text{LP}}) - H(z_{\text{LP}})z_{\text{LP}} + d, \quad k \in \mathbb{N}_0^{N-1}, \quad (9c)$$

$$\Delta u_k = u_k - u_{k-1}, \quad k \in \mathbb{N}_0^{N-1}, \quad (9d)$$

$$u_{\min} \leq u_k \leq u_{\max}, \quad k \in \mathbb{N}_0^{N-1}, \quad (9e)$$

$$y_{\min} \leq y_k \leq y_{\max}, \quad k \in \mathbb{N}_0^{N-1}, \quad (9f)$$

$$z_0 = \hat{z}(t), \quad d = \hat{d}(t), \quad u_{-1} = u(t - T_s), \quad (9g)$$

where the linearization point z_{LP} is the same as in the target optimization problem (8), so that a single linearization of the output mapping is shared by both problems at every sampling instant. The first-order Taylor expansion provides y_k , which enters the problem only through the output constraints (9f). In the objective function (9a), the deviation from the target is penalized directly in the lifted space by the weighting matrix Q_z , which we propose to derive as

$$Q_z = H(z_{\text{LP}})^\top Q_y H(z_{\text{LP}}). \quad (10)$$

In this way, the controller is still tuned through the matrix Q_y , which acts on the physical outputs and is therefore convenient to choose and directly comparable with the tuning of the standard formulation (4). The matrix Q_z constructed by (10) is positive semidefinite with rank at most n_y , and it is regularized by a small multiple of the identity matrix so that the MPC problem remains strictly convex. **TODO: The implementation uses $Q_z + 10^{-8}I$. State this value explicitly here, or keep the wording qualitative? The skill does not allow a number to be introduced without confirmation.** This is a significant change from the standard Koopman MPC, where the objective function is defined in the original space.

4.5. Closed-loop control structure

The closed-loop structure follows the one shown in Fig. 3, with three modifications: a linearization block is added, the observer is a time-varying

Kalman filter, and the two optimization problems are replaced by (8) and (9). The order in which the blocks are evaluated within one sampling period depends on the chosen linearization point. Algorithm 1 summarizes the loop for the linearization at the previous target.

Algorithm 1 Closed-loop control with nonlinear output mapping with linearization at previous target

Require: Koopman lifting function $g(\cdot)$, initial measurement $y(0)$

- 1: **Initialization:** $\hat{z}(0) \leftarrow g(y(0))$, $\hat{d}(0) \leftarrow 0$; solve (8) with $z_{\text{LP}} = \hat{z}(0)$ and $\hat{d}(0)$ to obtain $\bar{z}(-T_s)$
 - 2: **loop**
 - 3: **Measurement:** Obtain $y(t)$ from the plant
 - 4: **Linearization:** Set $z_{\text{LP}} \leftarrow \bar{z}(t - T_s)$ and compute $H(z_{\text{LP}})$ and $h(z_{\text{LP}})$
 - 5: **Observer:** Run the time-varying Kalman filter with $y(t)$, $u(t - T_s)$ and $H(z_{\text{LP}})$ to obtain the estimates $\hat{z}(t)$ and $\hat{d}(t)$
 - 6: **Target estimation:** Solve (8) with $\hat{d}(t)$ to obtain $\bar{z}(t)$, $\bar{u}(t)$, $\bar{y}(t)$
 - 7: **MPC:** Solve (9) with $\hat{z}(t)$, $\hat{d}(t)$, $\bar{z}(t)$ to obtain the optimal first input u_0^*
 - 8: **Apply input:** Apply $u(t) = u_0^*$ to the plant
 - 9: **end loop**
-

where the encoder $g : \mathbb{R}^{n_y} \rightarrow \mathbb{R}^{n_z}$ provides the initial lifted-state estimate from the first measurement, and the disturbance estimate is initialized at zero. Because the linearization point is the target of the previous sampling instant, the output mapping is linearized before the observer is run, and the same Jacobian is then used by the observer, by the target optimizer, and by the MPC problem. Algorithm 2 summarizes the loop for the linearization at the current state estimate.

Algorithm 2 Closed-loop control with nonlinear output mapping (linearization at current state estimate)

Require: Koopman lifting function $g(\cdot)$, initial measurement $y(0)$

- 1: **Initialization:** $\hat{z}(0) \leftarrow g(y(0))$, $\hat{d}(0) \leftarrow 0$
 - 2: **loop**
 - 3: **Measurement:** Obtain $y(t)$ from the plant
 - 4: **Observer:** Run the time-varying Kalman filter with $y(t)$, $u(t - T_s)$ and $H(\hat{z}(t - T_s))$ to obtain the estimates $\hat{z}(t)$ and $\hat{d}(t)$
 - 5: **Linearization:** Set $z_{\text{LP}} \leftarrow \hat{z}(t)$ and compute $H(z_{\text{LP}})$ and $h(z_{\text{LP}})$
 - 6: **Target estimation:** Solve (8) with $\hat{d}(t)$ to obtain $\bar{z}(t)$, $\bar{u}(t)$, $\bar{y}(t)$
 - 7: **MPC:** Solve (9) with $\hat{z}(t)$, $\hat{d}(t)$, $\bar{z}(t)$ to obtain the optimal first input u_0^*
 - 8: **Apply input:** Apply $u(t) = u_0^*$ to the plant
 - 9: **end loop**
-

The two algorithms differ in the linearization point and, as a consequence, in the position of the linearization step within the sampling period. In Algorithm 1, a single Jacobian is evaluated at the target of the previous sampling instant and is shared by the observer, by the target optimizer, and by the MPC problem. In Algorithm 2, the target optimizer and the MPC problem use the Jacobian at the current state estimate, which becomes available only after the measurement update. The observer itself cannot use this Jacobian, since it is the quantity that the update produces, and its measurement equation is therefore linearized at the estimate of the previous sampling instant.

4.6. State estimation with a time-varying Kalman filter

The observer estimates the augmented state $\tilde{z}(t) = [z(t)^\top, d(t)^\top]^\top$ from the noisy output $y(t)$ and the applied input $u(t)$. The augmented linear dynamics are identical to (1) with the original state replaced by the lifted Koopman state, resulting in

$$\tilde{z}(t + T_s) = \tilde{A} \tilde{z}(t) + \tilde{B} u(t), \quad \tilde{A} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix}, \quad \tilde{B} = \begin{bmatrix} B \\ 0 \end{bmatrix}, \quad (11)$$

where $\tilde{z}(t) \in \mathbb{R}^{n_z+n_d}$ is the augmented state stacking the lifted Koopman state and the output disturbance, and $\tilde{A} \in \mathbb{R}^{(n_z+n_d) \times (n_z+n_d)}$ and $\tilde{B} \in \mathbb{R}^{(n_z+n_d) \times n_u}$ are the augmented system matrices.

Because the output map is nonlinear, the measurement equation is re-linearized at each sampling instant around the linearization point z_{LP} selected by Algorithm 1 or Algorithm 2, which gives

$$y(t) \approx \tilde{C}(t) \tilde{z}(t) + c(t), \quad \tilde{C}(t) = [H(z_{\text{LP}}), C_d], \quad c(t) = h(z_{\text{LP}}) - H(z_{\text{LP}}) z_{\text{LP}}, \quad (12)$$

where $\tilde{C}(t) \in \mathbb{R}^{n_y \times (n_z + n_d)}$ is the time-varying output matrix formed by stacking the Jacobian of h at the linearization point and the disturbance output matrix C_d , and $c(t) \in \mathbb{R}^{n_y}$ is the linearization offset.

With the linear prediction model (11) and the time-varying output matrix (12), the filter runs the standard predict-update recursion at each sampling instant. The prediction step reads

$$\hat{\tilde{z}}^-(t) = \tilde{A} \hat{\tilde{z}}(t - T_s) + \tilde{B} u(t - T_s), \quad (13a)$$

$$P^-(t) = \tilde{A} P(t - T_s) \tilde{A}^\top + Q, \quad (13b)$$

where $\hat{\tilde{z}}^-(t) \in \mathbb{R}^{n_z + n_d}$ is the prior augmented-state estimate, $P^-(t) \in \mathbb{R}^{(n_z + n_d) \times (n_z + n_d)}$ is the prior error covariance, and $Q \succeq 0$ is the process noise covariance matrix. The superscript minus sign distinguishes the quantities available before the measurement $y(t)$ has been processed. The update step is then given by

$$S(t) = \tilde{C}(t) P^-(t) \tilde{C}(t)^\top + R, \quad (14a)$$

$$K(t) = P^-(t) \tilde{C}(t)^\top S(t)^{-1}, \quad (14b)$$

$$\hat{\tilde{z}}(t) = \hat{\tilde{z}}^-(t) + K(t) \left(y(t) - \tilde{C}(t) \hat{\tilde{z}}^-(t) - c(t) \right), \quad (14c)$$

$$P(t) = \left(I - K(t) \tilde{C}(t) \right) P^-(t), \quad (14d)$$

where $S(t) \in \mathbb{R}^{n_y \times n_y}$ is the innovation covariance, $K(t) \in \mathbb{R}^{(n_z + n_d) \times n_y}$ is the Kalman gain, $\hat{\tilde{z}}(t)$ is the posterior augmented-state estimate whose upper n_z entries give $\hat{z}(t)$ and lower n_d entries give $\hat{d}(t)$, $P(t)$ is the posterior error covariance, and $R \succ 0$ is the measurement noise covariance matrix.

4.7. Block-diagonal deep Koopman transformation

During our work, we encountered several problems with suboptimal or infeasible solutions of the Koopman MPC problem solved by the Gurobi solver. To improve the behavior of the optimization, we propose to bring the matrix A into a block-diagonal structure. The transformation is applied

after the Koopman operator has been identified, and it amounts to finding a transformation matrix V such that

$$A^{\text{bd}} = V^{-1}AV, \quad (15)$$

where $V \in \mathbb{R}^{n_z \times n_z}$ is constructed from the eigenvectors of A and A^{bd} is the real block-diagonal form of A , in which the eigenvalues are grouped into one-by-one blocks for the real eigenvalues and two-by-two blocks for the complex conjugate pairs. The remaining parts of the model have to be modified accordingly, and the same matrix V is used to transform the input and output matrices as well as the lifted state, which gives

$$B^{\text{bd}} = V^{-1}B, \quad C^{\text{bd}} = CV, \quad z_k^{\text{bd}} = V^{-1}z_k. \quad (16)$$

The Jacobian of the output mapping is transformed in the same way, resulting in

$$H^{\text{bd}}(z_k) = H(z_k)V = H(Vz_k^{\text{bd}})V. \quad (17)$$

For a fair comparison, this transformation is applied to every model identified in this work, not only to the Koopman models. The effects of the transformation are discussed in Sec. 7.

5. Identification

In this section, we introduce the benchmark process, the data collected from it, and the three prediction models whose closed-loop performance is compared in Sec. 7. All models are identified from the same data set with the same scaling.

5.1. Reactor-separator process

The considered plant consists of two continuous stirred tank reactors in series followed by a flash separator, with a recycle stream from the separator back to the first reactor. The plant structure is the one of Liu et al. (2009) and the kinetic, thermodynamic, and geometric parameters are those reported by Li and Swartz (2019). Two consecutive first-order reactions $A \rightarrow B \rightarrow C$ take place in both reactors, where B is the desired product and C an undesired by-product. The plant is therefore a selectivity problem, since additional heat increases the conversion of the reactant A but also the loss of the product B into C .

[grok4.7]The plant has nine states, namely the mass fractions of A and B and the temperature in each of the three vessels, and the third vessel is the separator, in which no reaction occurs. The manipulated variables are the three heat duties Q_1 , Q_2 , Q_3 and the fresh feed flow rate F_{10} to the first reactor. The fresh feed flow rate F_{20} to the second reactor and the recycle flow rate F_r are held fixed at the nominal values $0.5 \text{ m}^3/\text{s}$ and $4.0 \text{ m}^3/\text{s}$. The measured and controlled outputs are the three vessel temperatures T_1 , T_2 , T_3 and the bottoms quality x_{B3} , which is the mass fraction of B leaving the separator. The vessel inventories are constant, so the internal flow rates follow from F_{10} together with two fixed flows. The nominal operating point and the box within which the manipulated inputs are moved during identification are listed in Table 1. [/grok4.7]

Table 1: Manipulated variables of the reactor–separator plant, with the nominal operating point of Li and Swartz (2019) and the box used for the identification experiment.

Variable	Description	Nominal	Range
Q_1, Q_2, Q_3 [kJ/s]	Heat duties of the three vessels	10.0	[0.0, 25.0]
F_{10} [m^3/s]	Fresh feed to the first reactor	8.3	[3.0, 16.0]

Two mechanisms make the plant nonlinear. The first one is the Arrhenius dependence of both reaction rates on the vessel temperatures, and the second one is the relative-volatility flash of the separator, which makes the composition of the recycle stream a nonlinear function of the composition in the separator and closes a nonlinear loop back to the first reactor. As a consequence, the bottoms quality is not monotone in the supplied heat: the yield of B reaches a peak and then decreases again as more of the product is converted into C. A model identified around a single operating point can therefore reproduce only one branch of this behavior, which is the reason for choosing this plant as the benchmark. The plant equations are integrated in Python with the explicit Runge–Kutta solver of SciPy, and the sampling period used for identification and for control is $T_s = 1 \text{ s}$.

5.2. Identification data

The excitation is a sequence of 100 piecewise-constant input steps, each held for 180 s, which gives 18 000 samples of input–output data. All four manipulated inputs are stepped within the box of Table 1. Over the whole data set the temperatures cover the range from 326 K to 396 K and the bottoms quality the range from 0.40 to 0.76.

Zero-mean Gaussian measurement noise is added to all four outputs, with a standard deviation equal to 1% of the range of the respective output, that is 0.487 K, 0.527 K, and 0.610 K for the three temperatures and 0.0034 for the bottoms quality. The data set is split into the first 12 960 samples used for training, the following 2880 samples used for validation and early stopping, and the remaining 2160 samples, which are never seen during identification and serve as the test data. Inputs and outputs are standardized to zero mean and unit variance using the statistics of the training split only, and all three models are identified in these coordinates, to even the different scales of each variable.

5.3. Subspace model

The baseline model is identified with the N4SID algorithm Van Overschee and De Moor (2012) as implemented in the SIPPY package Armenise et al. (2018). This identification process was chosen as the industrial standard. The model order, the past and future horizons, the spectral radius, and the identification time on a Mac mini with an Apple M4 Pro processor are TBA. **TODO: Replace TBA with the 4×4 N4SID order, past and future horizons, spectral radius, and runtime on the named machine. The six-input values (order 17, horizons 57, spectral radius 0.9897, 2.6 s) must not be copied in.**

5.4. Koopman models

Two Koopman models are trained with the loss (6) on the same data. The first one restricts the decoder to a single matrix and thus provides the LTI model required by the standard offset-free Koopman MPC of Sec. 3. The second one keeps the nonlinear decoder and is the model used by the proposed scheme. For each

model, the lifted dimension, the network widths, the number of trainable parameters, and the spectral radius are TBA.

Both models are trained with the Adam optimizer in PyTorch Paszke et al. (2019) through the Neuromancer library Drgoňa et al. (2023). The minibatch size, the training-window length, the epoch budget, the learning rates, the loss weights of (6), and the training times on the same Mac mini with an Apple M4 Pro processor are TBA. As stated in Sec. 4.7, all three models are transformed into the real block-diagonal form before they are used in any control problem. **TODO: Replace each TBA in this subsection with the 4×4 Koopman configuration: lifted dimensions, widths, parameter counts, spectral radii, minibatch size, window length, epoch budget, learning rates, loss weights, and runtimes on the named machine. Do not reuse the six-input values.**

5.5. Open-loop accuracy on the test data

To compare the models on the data that none of them has seen, each model is initialized from the first sample of the test split and then run in open loop over the whole test window, driven only by the recorded inputs and without any state correction. The prediction errors are evaluated against the noise-free outputs of the plant, which are available because the plant is simulated, so that the reported numbers reflect the model error alone and are not inflated by the measurement noise. Fig. 4 shows the first four input steps of this window and Table 2 summarizes the accuracy over the whole window.

Table 2: Open-loop prediction accuracy on the test data, measured by the mean absolute error (MAE) against the noise-free plant outputs. The last column is the sum of the mean absolute errors of the four standardized outputs, which is the criterion on which the models are directly comparable.

Model	T_1 [K]	T_2 [K]	T_3 [K]	x_{B3} [-]	Σ MAE [-]
N4SID	TBA	TBA	TBA	TBA	TBA
Koopman, linear $\mathbf{h}$	TBA	TBA	TBA	TBA	TBA
Koopman, nonlinear $\mathbf{h}$	TBA	TBA	TBA	TBA	TBA

The numerical comparison on this test window is TBA. **TODO: Fill Table 2 from the 4×4 test split and restore the comparison**

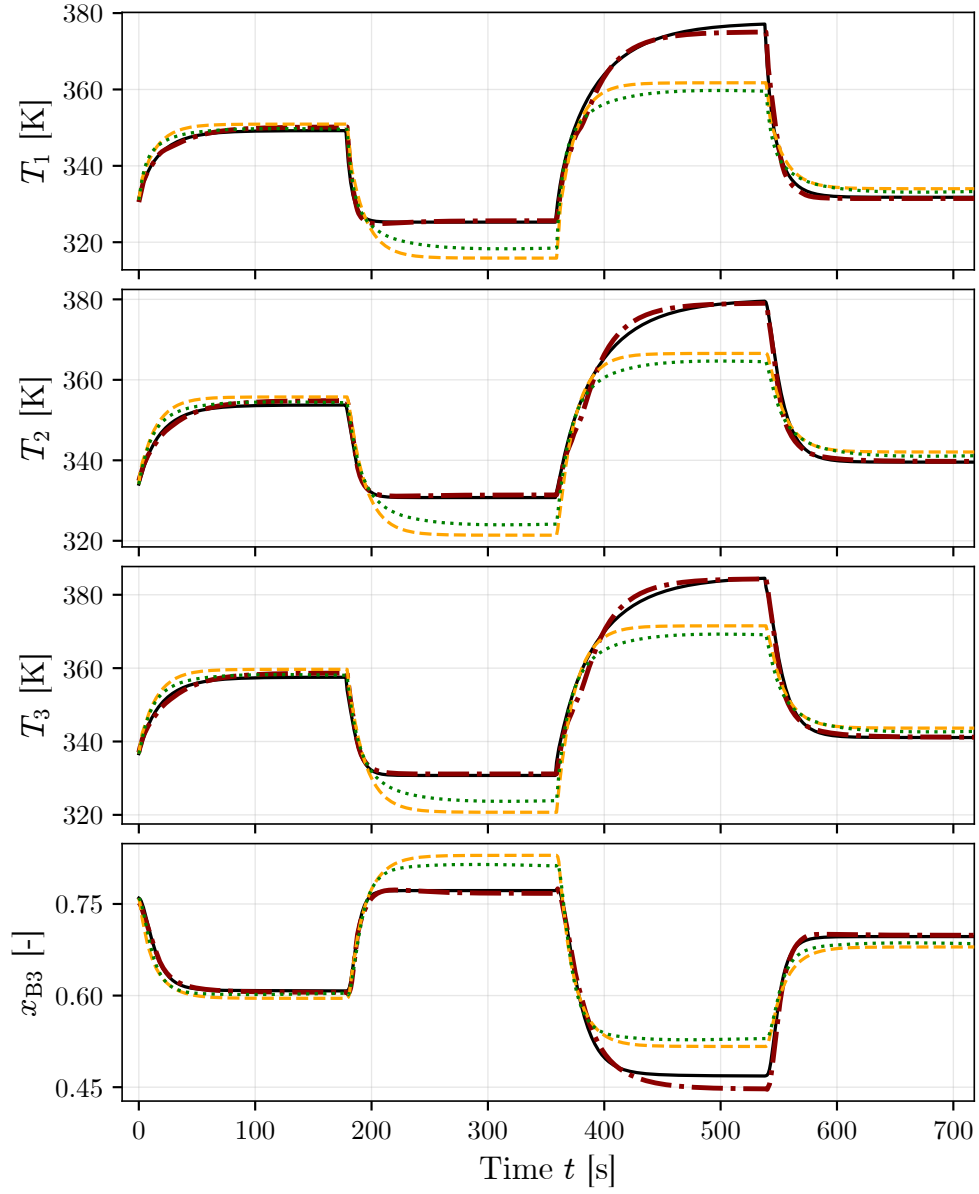


Figure 4: Open-loop predictions of the identified models on the first four input steps of the test data. The black solid line is the noise-free plant, the dash-dotted red line is the Koopman model with the nonlinear decoder, the dashed orange line is the Koopman model with the linear decoder, and the dotted green line is the N4SID model.

with the N4SID model. Fig. 4 is still the six-input test and has to be redrawn for the four manipulated inputs. Do not reuse the six-input errors or the statements that the nonlinear decoder reduces the summed error by about 85% and that the linear decoder is about 18% worse.

The Koopman model with the linear decoder is retained in the comparison, because it is the model that the standard offset-free Koopman MPC formulation of Sec. 3 requires. Restricting the decoder to a matrix leaves an input–output map of an LTI system, for which subspace identification is a mature and well-tuned tool, and the pair therefore isolates the effect of the nonlinear output mapping proposed in Sec. 4.1 from the effect of the lifting itself.

6. Control setup

In this section, we describe the closed-loop scenario on which the models of Sec. 5 are compared, together with the tuning, the observer, and the benchmark against which every controller is measured. The scenario, the weights, the constraints, and the disturbance sequence are identical in all runs reported in Sec. 7, so that the differences between the controllers are attributable to the prediction model and to the way the output mapping enters the control problem, and not to the way the loop was set up.

All four measured outputs T_1 , T_2 , T_3 , and x_{B3} are controlled, and the four manipulated variables of Table 1 are available to the controller, so the plant is square and the output specifications determine the steady-state target. The sampling period is the one used for identification, $T_s = 1$ s, and the prediction horizon is $N = 60$ samples. The input constraints (9e) are the identification box of Table 1, so that the controller never drives the plant outside the region from which the models were identified. The output constraints (9f) are an operating window rather than equipment limits, namely $310 \text{ K} \leq T_i \leq 380 \text{ K}$ for the three vessel temperatures and $0.35 \leq x_{B3} \leq 0.85$ for the bottoms quality. All signals, weights, and bounds are expressed in the standardized coordinates of Sec. 5, so that the three temperatures and the mass fraction contribute to the objective function on a comparable scale.

6.1. Grade campaign and reference changes

The scenario is a grade campaign on the bottoms quality, which is the specification that the separator has to deliver. The plant starts at the nominal steady state of Li and Swartz (2019) and the setpoint is held there for the first 350 s, so that the observer of every controller converges from its initialization before the first reference change. At $t = 350$ s the campaign switches to a premium grade, in which the mass fraction of the product B leaving the separator rises from 0.637 to 0.702, and at $t = 1000$ s the plant returns to the nominal grade. The whole simulation lasts 1500 s.

Both grades are steady states of the plant, obtained by simulating it to steady state at the nominal input of Table 1 and at the premium input [6.619, 8.544, 8.499, 11.224], and both are reachable inside the identification box, so that no reference change asks for an operating point that the models were never shown. The setpoints of the three temperatures are the corresponding steady-state values, 336.7, 344.1, and 346.2 K at the nominal grade and 331.5, 338.6, and 340.1 K at the premium grade, which means that the controller has to move the whole operating point and not the quality alone. The premium grade is reached with less heat and a larger fresh feed, because the yield of B is not monotone in the supplied heat, and the three temperatures consequently fall by approximately 6 K instead of rising. This is the direction in which a model identified around a single operating point is most likely to fail, and the grade change is chosen for exactly this reason.

6.2. Unmeasured feed disturbances

Two unmeasured feed upsets are applied to the plant while the premium grade is held. At $t = 600$ s the temperature of the fresh feed drops from 313 K to 311 K, which represents a drift of the feed preheater or of the utility that supplies it. At $t = 850$ s the fresh feed becomes leaner, with the mass fraction of the reactant A falling from 1.00 to 0.98, which represents a switch of the feed tank or a carry-over of an inert. Both upsets act on the plant only, and the encoder, the observer, the target optimizer, and the MPC problem are left unchanged, so that the four controllers built on an identified model can reject them only through the output disturbance estimate $\hat{d}(t)$.

The magnitudes are deliberately modest, because both grades have to remain reachable inside the identification box after the upsets. The measurements are noise-free in all closed-loop runs, although the models were identified from noisy data as described in Sec. 5. This is a deliberate choice, because the differences between the compared controllers are of the same order as the spread that a single noise realization introduces, and reporting one realization would attribute to the formulation what belongs to the noise.

6.3. Tuning

The target optimization (8) uses $Q_y^t = \text{diag}([1, 1, 1, 20])$, so that the steady-state target is the reachable operating point closest to the requested grade, with the bottoms quality weighted an order of magnitude above the temperatures. The MPC problem (9) is tuned in the original setup by

$$\begin{aligned} Q_y &= \text{diag}([2, 2, 2, 15]), & Q_{\Delta u} &= \text{diag}([0.5, 0.5, 0.5, 1]), \\ Q_u &= \text{diag}([0.2, 0.2, 0.2, 0.5]), \end{aligned} \quad (18)$$

where the entries of Q_y follow the order T_1, T_2, T_3, x_{B3} of the outputs and the entries of $Q_{\Delta u}$ and Q_u the order Q_1, Q_2, Q_3, F_{10} of the inputs. The bottoms quality again carries the largest output weight, because it is the variable that the campaign has to meet, while the temperatures are regulated but act mainly as operating limits. The input target towards which Q_u pulls is the nominal utility and throughput setting of Table 1, held constant over the whole campaign, so that this term expresses the economic preference of the plant rather than a tracking requirement.

Three tunings are reported. The first one is the original setup (18). In the second one, Q_y is multiplied by two and everything else is left unchanged, and in the third one, Q_y is divided by two and everything else is left unchanged, so that the relative weight of the outputs against the inputs is moved by a factor of two in each direction. The purpose of this sweep is not to search for a good controller but to test the mechanism of the proposed formulation. The proposed scheme differs from the standard one only in the way the output mapping is used, in the objective function through (10) and in the output constraints through (9c). Its advantage is therefore expected to grow with the weight that the objective function

places on the outputs, and to shrink when Q_y is reduced and the outputs become a secondary concern.

6.4. Observers and the benchmark

Each controller is paired with the observer that its formulation requires. The two controllers built on an LTI model, namely the one using the subspace model and the one using the Koopman model with the linear decoder, use a stationary Kalman filter on the augmented system (1), while the proposed controllers use the time-varying filter of Sec. 4.6. The number of estimated disturbances equals the number of measurements, $n_d = n_y = 4$, with $B_d = 0$ and $C_d = I$, so that the disturbance is purely additive on the outputs and the rank condition (2) is satisfied. The covariances in the standardized coordinates are $P(0) = I$, $R = 0.3 I$, and $Q = \text{diag}([0.1 I, 0.1 I])$, with the two blocks acting on the lifted state and on the disturbance, respectively. The only exception is the subspace baseline, whose disturbance block had to be raised to $1.0 I$ to obtain an acceptable closed-loop response; a larger disturbance covariance produces a faster estimate of $\hat{d}(t)$, so this exception favors the baseline.

The benchmark is a nonlinear MPC that uses the plant equations themselves as the prediction model. It is paired with an extended Kalman filter and with a nonlinear steady-state target selector built on the same equations, both discretized by a fourth-order Runge–Kutta map with ten substeps per sampling period, so that the observer of the benchmark is not biased with respect to the plant stepper. Furthermore, the benchmark is informed of the new feed conditions at the instant at which each upset is applied, so that it suffers neither from process-model mismatch nor from a lag in the estimation of the disturbance. It uses the same horizon, the same weights, the same constraints, and the same references as every other controller. Its closed-loop performance is therefore the practical bound of this comparison, and we use it as the normalization of the results rather than as a competitor.

To evaluate the closed-loop performance, we accumulate over the whole simulation the same quantities that the MPC problem

penalizes, which gives

$$J = \underbrace{\sum_{t \in \mathcal{T}} \|y(t) - y_{\text{sp}}(t)\|_{Q_y}^2}_{J_y} + \underbrace{\sum_{t \in \mathcal{T}} \|u(t) - u(t - T_s)\|_{Q_{\Delta u}}^2}_{J_{\Delta u}} + \underbrace{\sum_{t \in \mathcal{T}} \|u(t) - \bar{u}(t)\|_{Q_u}^2}_{J_u}, \quad (19)$$

where J is the closed-loop objective, J_y , $J_{\Delta u}$, and J_u are its three parts, $\mathcal{T} = \{0, T_s, \dots, (n_{\text{sim}} - 1)T_s\}$ is the set of the sampling instants of the campaign, $n_{\text{sim}} = 1500$ is its length in samples, and $\|v\|_Q^2 = v^\top Q v$. The three parts are the closed-loop counterparts of the three terms of (4a) and are evaluated with the weights of the tuning that is being reported. A large J_y means that the plant spends time away from the requested grade, which in a campaign is off-specification product, whereas a large $J_{\Delta u}$ or J_u means that the duties and the flow rates are moved far and held far from the economically preferred operating point, which is paid for in utilities and in throughput.

The target optimization (8) and the MPC problem (9) are quadratic programs, formulated in CVXPY and solved with Gurobi Gurobi Optimization, LLC (2024) restricted to a single thread. The decoder $h(z_{\text{LP}})$ and its Jacobian $H(z_{\text{LP}})$ are evaluated in PyTorch Paszke et al. (2019) at every sampling instant. The nonlinear benchmark and its target selector are formulated in CasADi through the do-mpc toolbox and solved with IPOPT using the MUMPS linear solver, and the plant itself is integrated by the explicit Runge–Kutta solver of SciPy. All closed-loop runs were carried out on a Mac mini with an Apple M4 Pro processor. **TODO: add bibliography entries for CVXPY, CasADi, do-mpc, IPOPT, and SciPy**

7. Closed-loop results

In this section, we present and analyze the closed-loop results obtained in the scenario of Sec. 6. Four controllers are compared against the nonlinear benchmark, namely the offset-free MPC of Sec. 3 using the subspace model, the same formulation using the Koopman model with the linear decoder, and the proposed formulation of Sec. 4 in its two variants, linearized at the target of

the previous sampling instant as in Algorithm 1 and at the current state estimate as in Algorithm 2.

7.1. The effect of block-diagonal transformation

After this transformation we are able to enhance the performance in optimization as can be seen in Table 3. This table is done using the formulation of Koopman MPC which can be seen in Sec. 3. The table shows the decrease in a computational times. Also with this formulation we are able to obtain optimal solution TBA of the time during simulation. Contrary, the full Koopman MPC formulation is able to obtain optimal solution only TBA of the time with the rest of the time giving warning, that the solution is sub-optimal. We can see also increase in a control performance, which is measured by the objective function value, which is in optimal case around TBA lower. Also the settling time for both levels is lower. **TODO: Fill Table 3 from the**

Table 3: Comparison of structure of problem to solver time and control performance.

Structure	Time TE/MPC	Optimal? [%]	OBJ	ST h_1	ST h_2
Full	TBA	TBA	TBA	TBA	TBA
Block Diagonal	TBA	TBA	TBA	TBA	TBA

4×4 runs: solver times, the fraction of optimal solutions, the objective, and both settling times. The paragraph above still marks those quantities TBA, and the six-input entries must not be copied back.

The second option how to obtain the block diagonal structure is to learn the Koopman operator directly in the block diagonal form. This can be done by limiting the number of learnable parameters in a Koopman matrix during training to those that are in the block diagonal form. This also speeds the training as less parameters are learned.

To obtain an optimal performance, which is necessary for the real time control, we are proposing to use the block diagonal structure of the Koopman operator. There is no necessity to use the full Koopman operator, as the block diagonal structure is able to capture the same dynamics of the system and favors the optimization performance. The way this structure is obtained, during or after the training, is not important, as long as the Koopman operator is in the block diagonal form.

7.2. Closed-loop performance of the grade campaign

[grok4.6high]Fig. 5 shows the controlled outputs over the whole campaign for the original setup (18), Fig. 6 shows the manipulated variables, and Tables 4–6 report the closed-loop objective (19) and its three parts for the three tunings of Sec. 6. [/grok4.6high]

[grok4.6high]As can be seen in Fig. 5, every plotted controller tracks both grade changes, and the steady-state offset in x_{B3} is removed after each unmeasured feed upset. The nonlinear benchmark, which is informed of the new feed conditions, barely moves the temperatures after the upsets, whereas the identified-model controllers show a temperature excursion of approximately 1 K after the drop in the feed temperature and a smaller one after the feed becomes leaner. The subspace controller retains a temperature offset of a few tenths of a kelvin during the holds, which both proposed linearizations remove. The two proposed series nearly coincide, except at the grade-change transients. [/grok4.6high]

[grok4.6high]As can be seen in Fig. 6, the subspace controller departs more from the nonlinear benchmark on the manipulated variables. Both proposed linearizations stay close to the nonlinear benchmark, closer than the MPC that uses the Koopman model with the linear decoder, and none of the plotted inputs leaves the identification box of Table 1. [/grok4.6high]

[grok4.6high]Fig. 7 shows the four components of the output disturbance estimate $\hat{d}(t)$ for the linear MPCs. After each grade change the two LTI controllers accumulate several kelvin of correction on the temperatures, which is the amount by which those models miss the new steady state of the plant. The proposed controllers keep a smaller estimate over the same intervals, and the two linearization points nearly coincide except at the second grade change. Both feed upsets produce a further jump in $\hat{d}(t)$, since the observer is the only path through which those plant changes enter the control problem. [/grok4.6high]

TODO: The disturbance panels are the stored scaled $\hat{d}(t)$ multiplied by the output scaler, so the units are K and mass fraction. Switch the figure to standardized coordinates if that is the intended comparison.

Two comparisons are drawn from the tables. The first one is the comparison of each controller with the nonlinear benchmark,

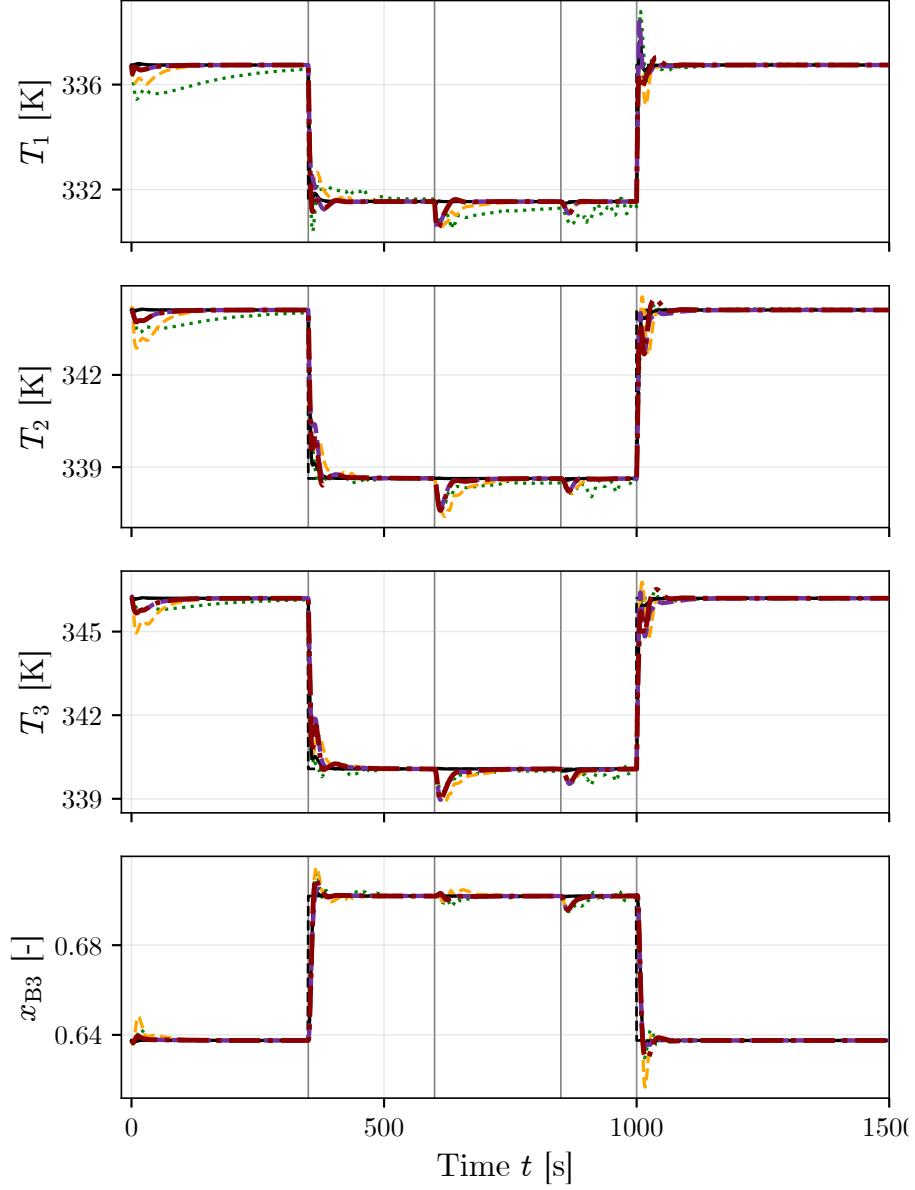


Figure 5: [grok4.6high]Closed-loop outputs of the grade campaign in the original setup. The panels are T_1 , T_2 , T_3 , and x_{B3} . The black dashed line is the setpoint, the solid black line is the nonlinear benchmark, the dotted green line is the MPC using the N4SID model, the dashed orange line is the MPC using the Koopman model with the linear decoder, the dashed purple line is the proposed formulation linearized at the current state estimate, and the dash-dotted red line is the proposed formulation linearized at the target of the previous sampling instant. The grey vertical lines mark the two grade changes at 350 s and 1000 s and the two unmeasured feed upsets at 600 s and 850 s. [grok4.6high]

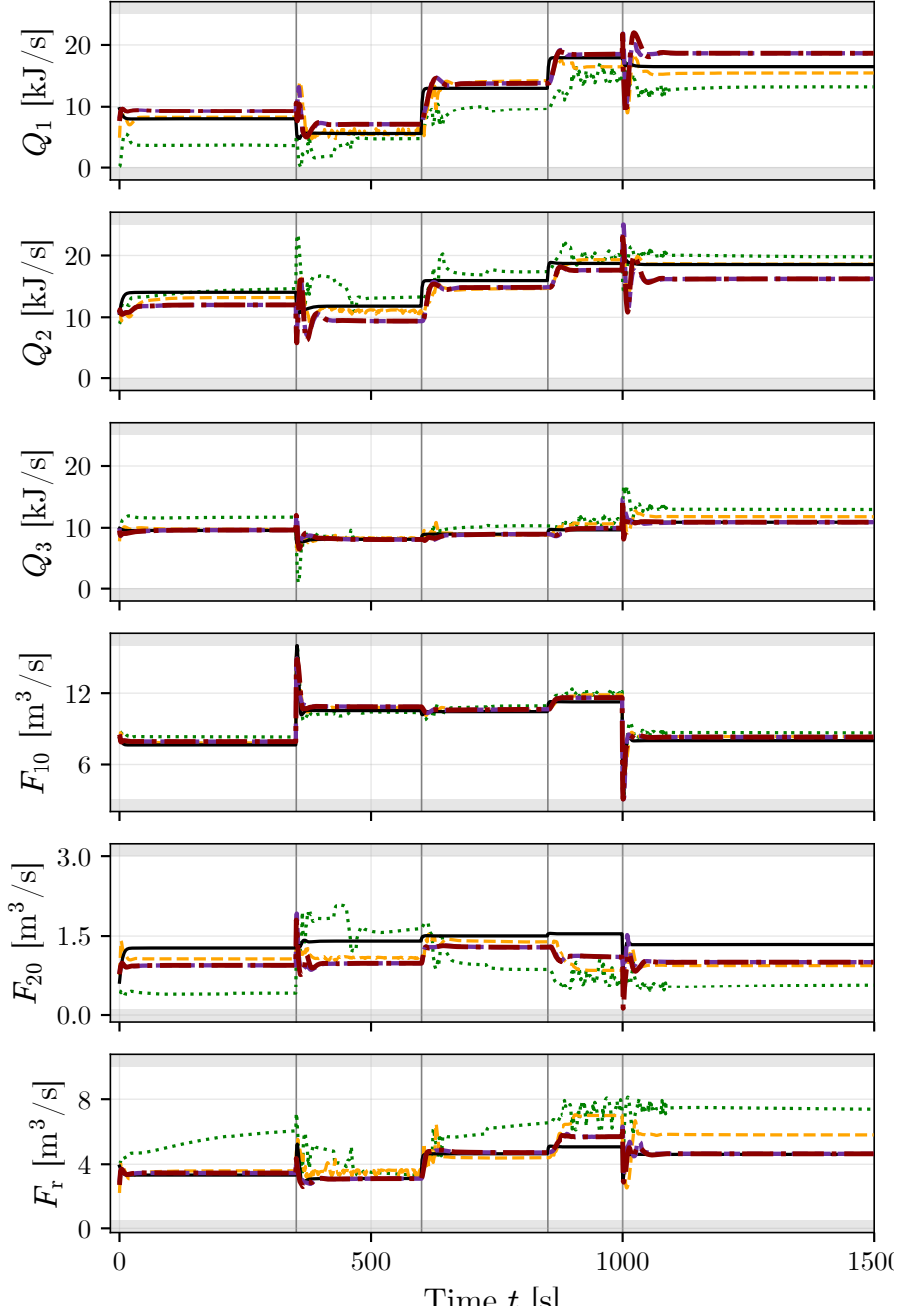


Figure 6: Closed-loop manipulated variables of the grade campaign in the original setup. The panels are Q_1 , Q_2 , Q_3 , and F_{10} . The solid black line is the nonlinear benchmark, the dotted green line is the MPC using the N4SID model, the dashed orange line is the MPC using the Koopman model with the linear decoder, the dashed purple line is the proposed formulation linearized at the current state estimate, and the dash-dotted red line is the proposed formulation linearized at the target of the previous sampling instant. The grey vertical lines mark the same grade changes and feed upsets as in Fig. 5, and the grey bands are the input bounds of Table 1.

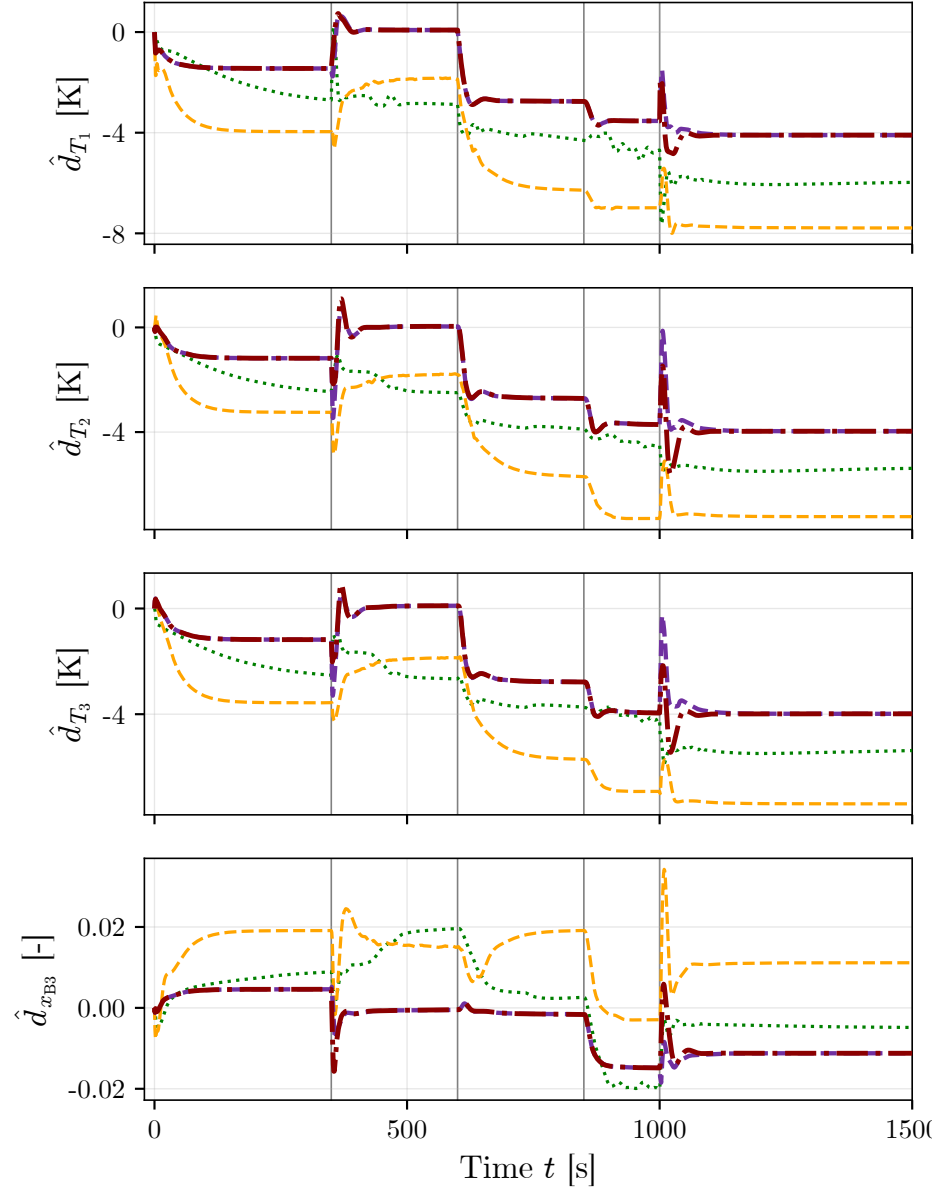


Figure 7: [grok4.6high]Output disturbance estimates $\hat{d}(t)$ of the four linear MPCs in the original setup, expressed in the physical output units. The four panels correspond to T_1 , T_2 , T_3 , and x_{B3} . The dotted green line is the MPC using the N4SID model, the dashed orange line is the MPC using the Koopman model with the linear decoder, the dashed purple line is the proposed formulation linearized at the current state estimate, and the dash-dotted red line is the proposed formulation linearized at the previous target. The grey vertical lines mark the same grade changes and feed upsets as in Fig. 5. [/grok4.6high]

Table 4: Closed-loop performance in the original setup (18). The three parts of the objective function are the ones defined in (19), and the last column is normalized so that **100%** stands for the performance of the nonlinear benchmark, which uses the exact plant model.

Controller	J_y	$J_{\Delta u}$	J_u	J	$J/J_{\text{NMPC}} [\%]$
Nonlinear MPC, exact model	TBA	TBA	TBA	TBA	TBA
N4SID	TBA	TBA	TBA	TBA	TBA
Koopman, linear $\mathbf{h}$	TBA	TBA	TBA	TBA	TBA
Proposed, $\mathbf{z}_{\text{LP}} = \bar{\mathbf{z}}(\mathbf{t} - \mathbf{T}_s)$	TBA	TBA	TBA	TBA	TBA
Proposed, $\mathbf{z}_{\text{LP}} = \hat{\mathbf{z}}(\mathbf{t})$	TBA	TBA	TBA	TBA	TBA

Table 5: Closed-loop performance with the output weight doubled, $\mathbf{Q}_y = \text{diag}([4, 4, 4, 30])$, and the remaining weights of (18) unchanged. The columns have the same meaning as in Table 4, and the objective function is accumulated with the weights of this tuning.

Controller	J_y	$J_{\Delta u}$	J_u	J	$J/J_{\text{NMPC}} [\%]$
Nonlinear MPC, exact model	TBA	TBA	TBA	TBA	TBA
N4SID	TBA	TBA	TBA	TBA	TBA
Koopman, linear $\mathbf{h}$	TBA	TBA	TBA	TBA	TBA
Proposed, $\mathbf{z}_{\text{LP}} = \bar{\mathbf{z}}(\mathbf{t} - \mathbf{T}_s)$	TBA	TBA	TBA	TBA	TBA
Proposed, $\mathbf{z}_{\text{LP}} = \hat{\mathbf{z}}(\mathbf{t})$	TBA	TBA	TBA	TBA	TBA

which quantifies how much of the achievable performance is lost when the plant equations are replaced by an identified model. The second one is the comparison of the proposed formulation with the standard offset-free Koopman MPC of Sec. 3, which isolates the effect of the nonlinear output mapping, since the two controllers share the scenario, the tuning, the constraints, and the disturbance sequence, and differ only in the model and in the way the outputs are recovered from the lifted state. The subspace baseline is reported as well, because it is the model that would be identified first for a plant of this kind.

TODO: Tables 4–6 are TBA because they still held the six-input campaign. Fill every cell from the 4×4 runs, then state each controller relative to the benchmark, the gap between the proposed formulation and the standard Koopman MPC, and the

Table 6: Closed-loop performance with the output weight halved, $Q_y = \text{diag}([1, 1, 1, 7.5])$, and the remaining weights of (18) unchanged. The columns have the same meaning as in Table 4, and the objective function is accumulated with the weights of this tuning.

Controller	J_y	$J_{\Delta u}$	J_u	J	$J/J_{\text{NMPC}} [\%]$
Nonlinear MPC, exact model	TBA	TBA	TBA	TBA	TBA
N4SID	TBA	TBA	TBA	TBA	TBA
Koopman, linear $\mathbf{h}$	TBA	TBA	TBA	TBA	TBA
Proposed, $\mathbf{z}_{\text{LP}} = \bar{\mathbf{z}}(\mathbf{t} - \mathbf{T}_s)$	TBA	TBA	TBA	TBA	TBA
Proposed, $\mathbf{z}_{\text{LP}} = \hat{\mathbf{z}}(\mathbf{t})$	TBA	TBA	TBA	TBA	TBA

position of the subspace baseline. Figs. 5–7 and the sentences that quote a temperature excursion of about 1 K are still the six-input campaign and have to be replaced with the same runs.

The three tunings answer whether the improvement is attributable to the output mapping. The proposed formulation replaces the linear recovery $\mathbf{y}_k = \mathbf{C}\mathbf{z}_k$ by the first-order expansion of the decoder, both in the weighting matrix (10) and in the output constraints (9c), so that its advantage acts on the part of the objective function that involves the outputs. The larger the share of J_y in the total, the larger the advantage should be, and the advantage should shrink when Q_y is halved and the accuracy of the predicted outputs influences the computed input less.

TODO: confirm or refute the expected trend across Tables 4–6, and report the normalized objective of the proposed formulation at the two extremes of the sweep

A separate question is which of the two linearization points is to be preferred. Algorithm 1 expands the output mapping at the target of the previous sampling instant, which is the point at which the target optimization is the most accurate, whereas Algorithm 2 expands it at the current state estimate, which is the point at which the first predicted output is the most accurate. The two points are far apart during the transient after a grade change and nearly coincide during the holds, so the difference between the two variants is expected to be visible in the transients only.

TODO: compare the two linearization points across the three tunings and state which one is recommended

8. Discussion

9. Conclusion

References

- Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M., 2019. Optuna: A next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 2623–2631.
- Armenise, G., Bacci di Capaci, R., Vaccari, M., Pannocchia, G., 2018. An open-source system identification package for multivariable processes, in: 2018 UKACC 12th International Conference on Control (CONTROL). doi:10.1109/CONTROL.2018.8516791.
- Drgoňa, J., Tuor, A., Koch, J., Shapiro, M., Vrabie, D., 2023. Neuro-MANCER: Neural Modules with Adaptive Nonlinear Constraints and Efficient Regularizations URL: <https://github.com/pnnl/neuromancer>.
- Gurobi Optimization, LLC, 2024. Gurobi Optimizer Reference Manual. URL: <https://www.gurobi.com>.
- Kaiser, E., Kutz, J.N., Brunton, S.L., 2021. Data-driven discovery of koopman eigenfunctions for control. Machine Learning: Science and Technology 2, 035023.
- Koopman, B.O., 1931. Hamiltonian systems and transformation in hilbert space. Proceedings of the National Academy of Sciences 17, 315–318.
- Korda, M., Mezić, I., 2018. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. Automatica 93, 149–160.
- Li, H., Swartz, C.L.E., 2019. Dynamic real-time optimization of distributed MPC systems using rigorous closed-loop prediction. Computers & Chemical Engineering 122, 356–371. doi:10.1016/j.compchemeng.2018.08.028.
- Liu, J., Muñoz de la Peña, D., Christofides, P.D., 2009. Distributed model predictive control of nonlinear process systems. AIChE Journal 55, 1171–1184.

- Lusch, B., Kutz, J.N., Brunton, S.L., 2018. Deep learning for universal linear embeddings of nonlinear dynamics. *Nature communications* 9, 4950.
- Mezić, I., 2005. Spectral properties of dynamical systems, model reduction and decompositions. *Nonlinear Dynamics* 41, 309–325.
- Pannocchia, G., Rawlings, J.B., 2003. Disturbance models for offset-free model-predictive control. *AIChE journal* 49, 426–437.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S., 2019. Pytorch: An imperative style, high-performance deep learning library, in: *Advances in Neural Information Processing Systems* 32, pp. 8024–8035.
- Rowley, C.W., Mezić, I., Bagheri, S., Schlatter, P., Henningson, D.S., 2009. Spectral analysis of nonlinear flows. *Journal of fluid mechanics* 641, 115–127.
- Van Overschee, P., De Moor, B., 2012. Subspace identification for linear systems: Theory—Implementation—Applications. Springer Science & Business Media.
- Williams, M.O., Kevrekidis, I.G., Rowley, C.W., 2015. A data-driven approximation of the koopman operator: Extending dynamic mode decomposition. *Journal of Nonlinear Science* 25, 1307–1346.